DS5899: Special Topics in Data Science

Leveraging NLP in Asset Management

Week 8: Reasoning

Instructor: Che Guan, Ph.D.

Part 1:

Chain-of-Thought (CoT)

System 1 and System 2 Thinking

- Human cognition seamlessly transitions between intuitive, heuristic (System 1) and analytical, deliberative (System 2) reasoning based on the context. However, LLMs do not possess this fluid adaptability. This lack of flexibility can result in rigid and inconsistent performance when encountering tasks that differ from their trained patterns.
- System 1-aligned models perform better in commonsense tasks. System 2-aligned models excel in arithmetic and symbolic reasoning.
- System 1 models employ more definitive answers, whereas System 2 models demonstrate greater uncertainty

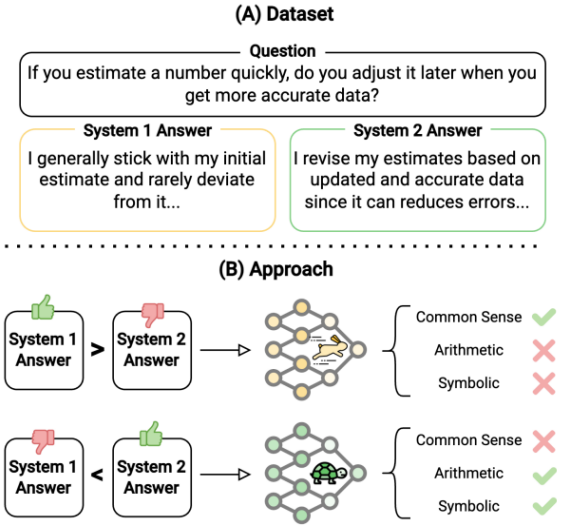
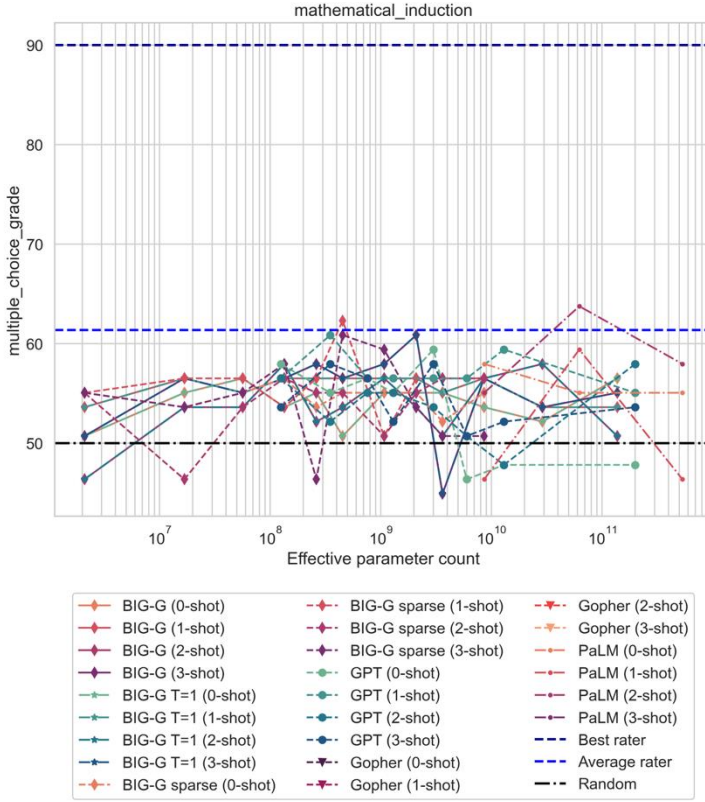


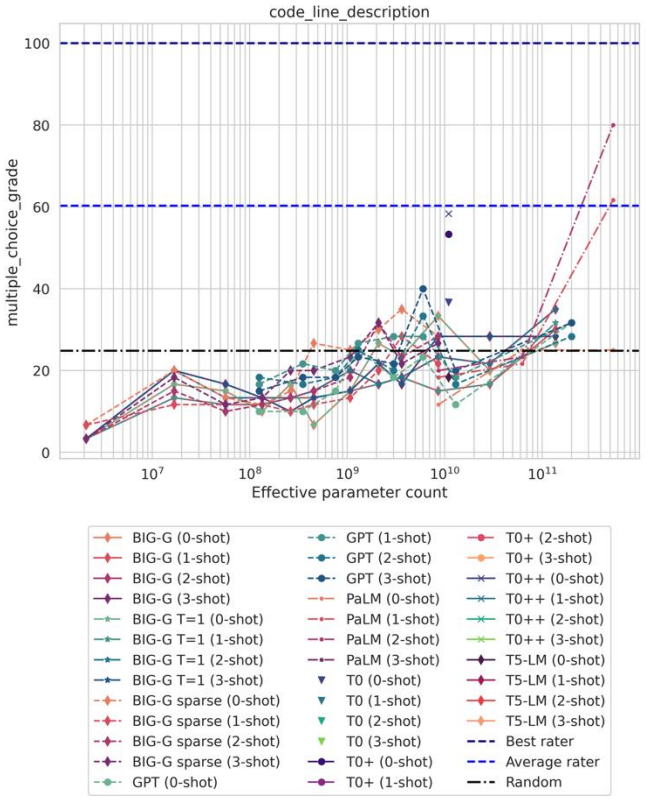
Figure 1: (A) Sample of dataset with System 1 and System 2 answers. (B) Overview of our approach for aligning LLMs with fast and slow thinking, highlighting performance gains across reasoning benchmarks.

Do You Observe the Scaling Law?

- This task tests the language model's capability to understand induction by asking the model to verify the correctness of an induction argument.



- The goal of this task is to test the ability of language models to understand computer programming



Chain-of-Thought Prompting

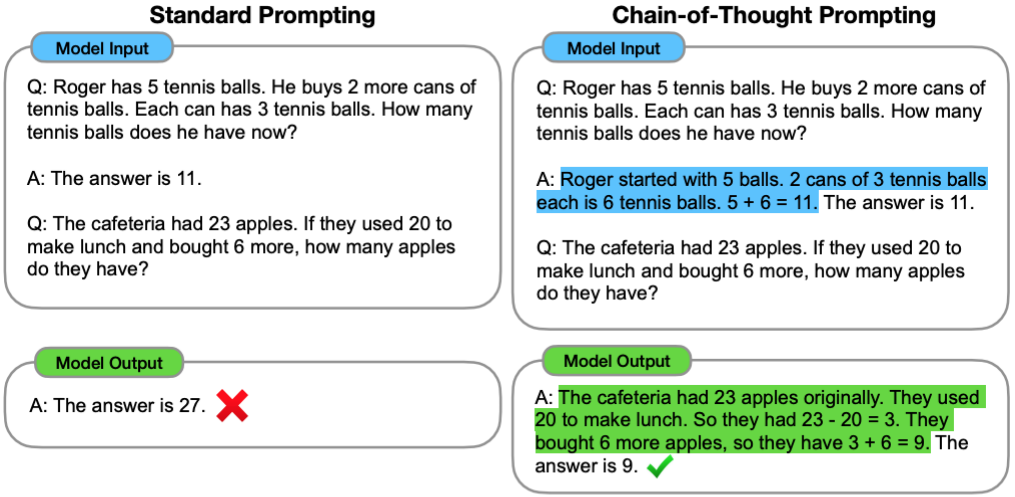


Figure 1: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

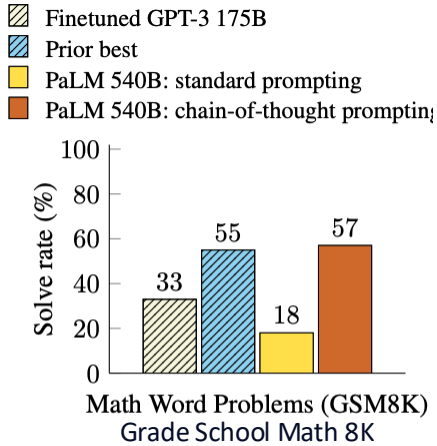


Figure 2: PaLM 540B uses chain-of-thought prompting to achieve new state-of-the-art performance on the GSM8K benchmark of math word problems. Finetuned GPT-3 and prior best are from Cobbe et al. (2021).

LLMs are Zero-Shot Reasoners

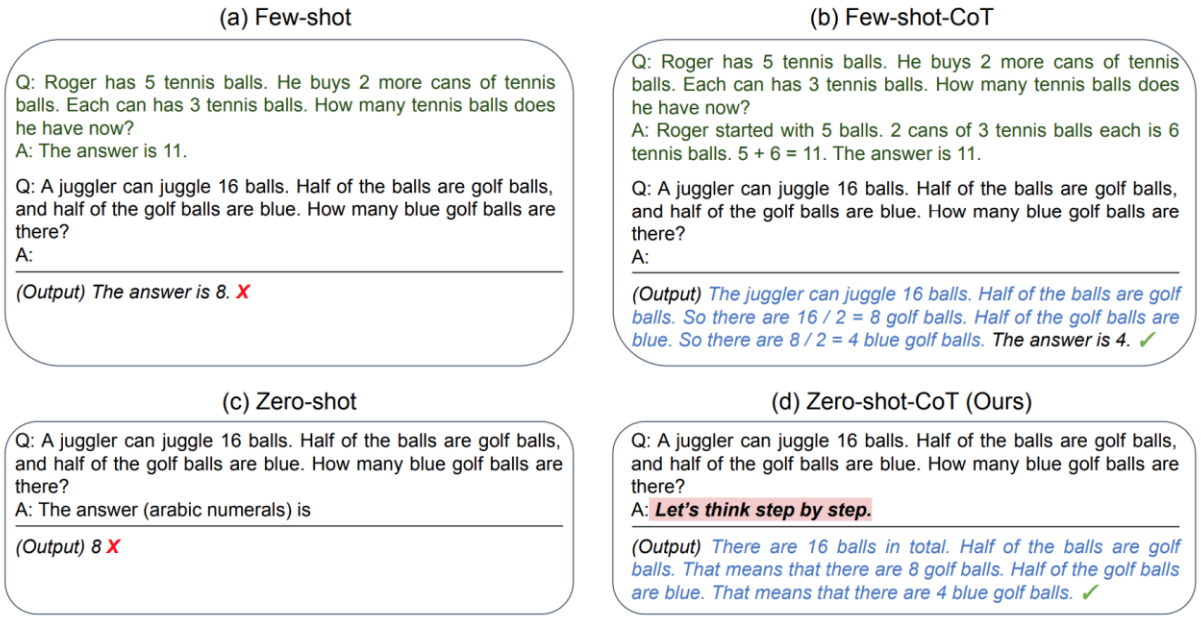
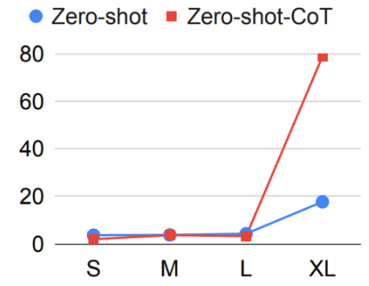
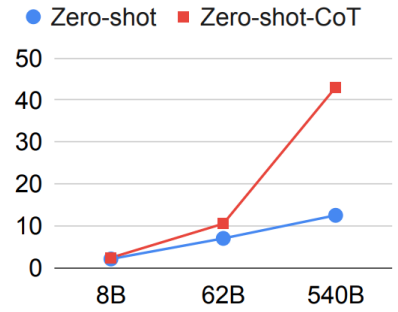


Figure 1: Example inputs and outputs of GPT-3 with (a) standard Few-shot ([Brown et al., 2020]), (b) Few-shot-CoT ([Wei et al., 2022]), (c) standard Zero-shot, and (d) ours (Zero-shot-CoT). Similar to Few-shot-CoT, Zero-shot-CoT facilitates multi-step reasoning (blue text) and reach correct answer where standard prompting fails. Unlike Few-shot-CoT using step-by-step reasoning examples **per task**, ours does not need any examples and just uses the same prompt "Let's think step by step" *across all tasks* (arithmetic, symbolic, commonsense, and other logical reasoning tasks).



(b) MultiArith on Instruct GPT-3

MultiArith: math word problems and a final answer



(c) GMS8K on PaLM

GSM8K: Grade School Math 8K

Challenging BIG-Bench tasks and Whether Chain-of-thought Can Solve Them

- BIG-Bench Hard (BBH) focus on a suite of 23 challenging BIG-Bench tasks

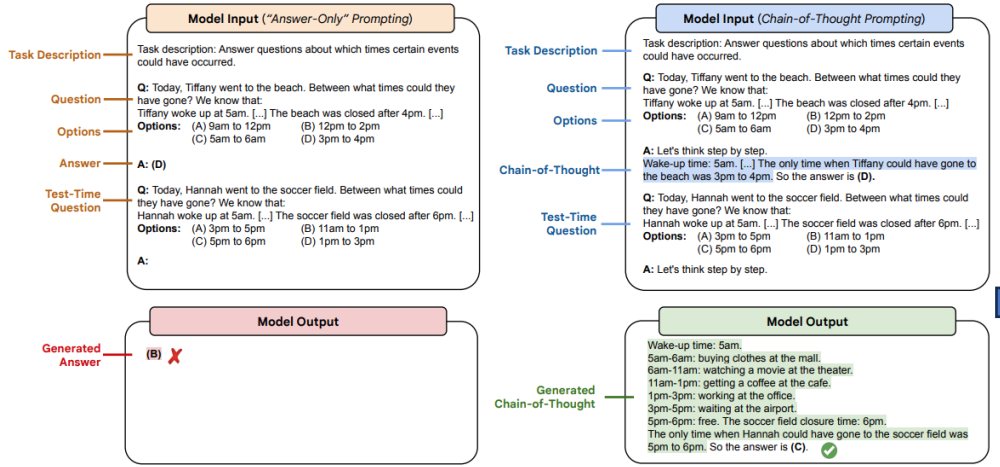


Figure 3: An illustration of the two prompting setups we explore in our paper (answer-only and CoT prompting). Both setups include task descriptions and options in the input prompt. The task here is *Temporal Sequences*.

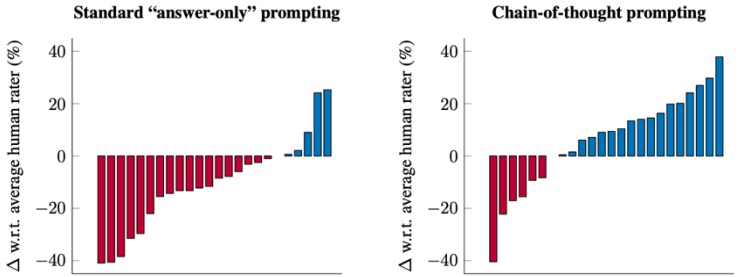


Figure 1: Per-task delta between Codex (code-davinci-002) and the average human-rater performance on 23 challenging tasks in BIG-Bench Hard, for standard *“answer-only”* (left) and *chain-of-thought* (right) prompting.

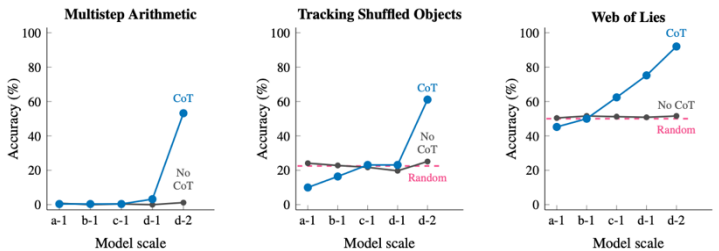
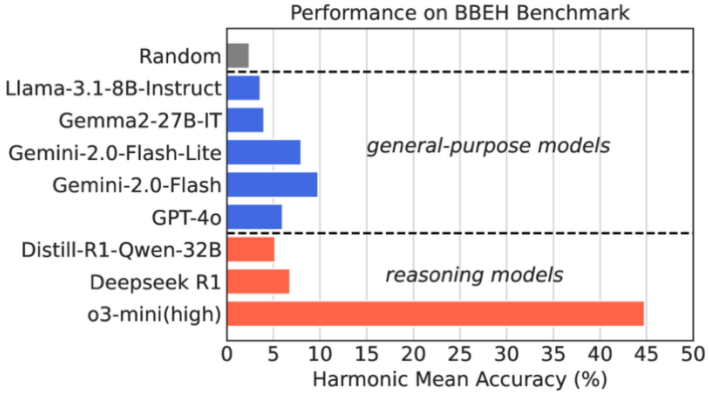


Figure 5: For three tasks where prompting without CoT has a flat scaling curve (Srivastava et al., 2022), CoT prompting enables emergent-task performance, which is performance that goes from random to substantially above random as scale increases. Scaling behavior of chain-of-thought (CoT) prompting on BIG-Bench Hard (BBH; 23 task unweighted average). OpenAI models are the following: a-1 (text-ada-001), b-1 (text-babbage-001), c-1 (text-curie-001), d-1 (text-davinci-001), and d-2 (text-davinci-002).

What is your insight from the flat scaling curve?

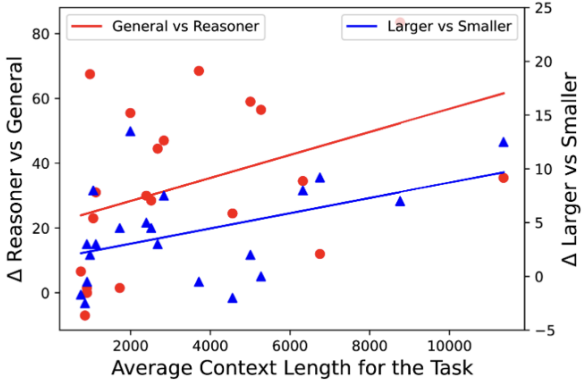
BIG-Bench Extra Hard

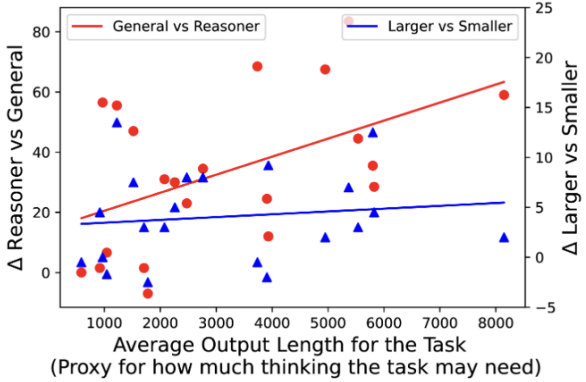
- Current LLM reasoning benchmarks predominantly focus on mathematical and coding abilities, leaving a gap in evaluating broader reasoning proficiencies.
- BIG-Bench dataset has served as a crucial benchmark for evaluating the general reasoning capabilities of LLMs. BBEH (BIG-Bench Extra Hard) replaces each task in BBH with a novel task that probes a similar reasoning capability but exhibits significantly increased difficulty.



BIG-Bench Extra Hard

- The gains of o3-mini tend to increase compared to GPT4o both when context length increases and when the required thinking increases, showing how reasoning models may have improved across both directions compared to general models.
- For Gemini 2.0 Flash vs Gemini 2.0 Flash-Lite, we see a similar increase in gains when the context length increases, but the curve for the case of increased thinking remains mostly flat.

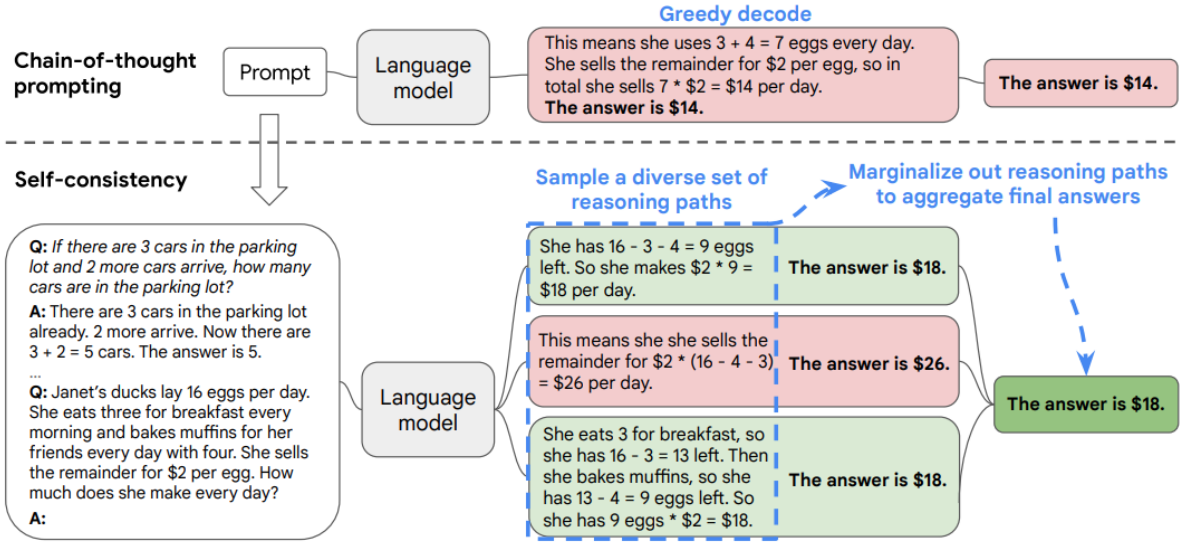




Why is the length of the output important?

What insights do you derive from the two comparisons?

Self-Consistency Improves CoT Reasoning in Language Models



	GSM8K	MultiArith
Greedy decode	56.5	94.7
Weighted avg (unnormalized)	56.3 ± 0.0	90.5 ± 0.0
Weighted avg (normalized)	22.1 ± 0.0	59.7 ± 0.0
Weighted sum (unnormalized)	59.9 ± 0.0	92.2 ± 0.0
Weighted sum (normalized)	74.1 ± 0.0	99.3 ± 0.0
Unweighted sum (majority vote)	74.4 ± 0.1	99.3 ± 0.0

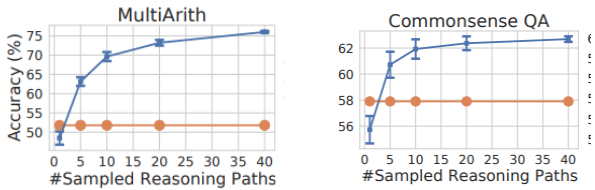
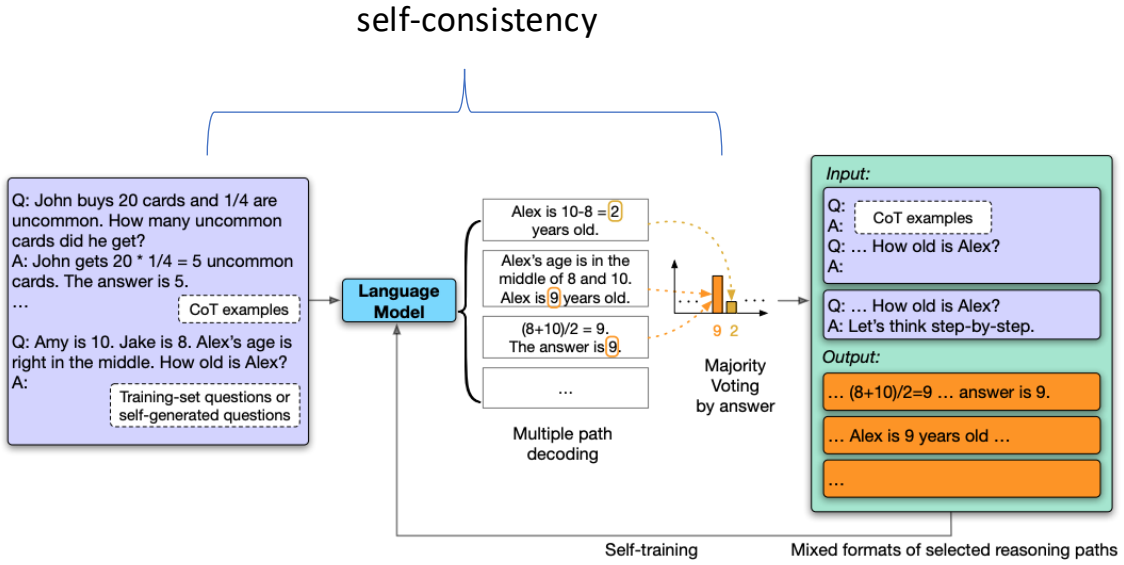


Figure 1: The self-consistency method contains three steps: (1) prompt a language model using chain-of-thought (CoT) prompting; (2) replace the "greedy decode" in CoT prompting by sampling from the language model's decoder to generate a diverse set of reasoning paths; and (3) marginalize out the reasoning paths and aggregate by choosing the most consistent answer in the final answer set.

- Self-consistency (blue) significantly improves accuracy over CoT-prompting with greedy decoding (orange) across arithmetic and commonsense reasoning tasks.
- Sampling a higher number of diverse reasoning paths consistently improves reasoning accuracy

Large Language Models Can Self-Improve



	Prompting Method	GSM8K
	Previous SOTA	82.3[a]
w/o **LMSI**	Standard-Prompting	17.9
	CoT-Prompting	56.5
	Self-Consistency	74.4
LMSI	Standard-Prompting	32.2
	CoT-Prompting	73.5
	Self-Consistency	82.1

	Results on **GSM8K**		
	8 billion	62 billion	540 billion
w/o **LMSI**	5.0	29.7	56.5
Distilled from **LMSI** 540 billion	33.4	57.4	-

Figure 1: Overview of our method. With Chain-of-Thought (CoT) examples as demonstration (Wei et al., 2022b), the language model generates multiple CoT reasoning paths and answers (temperature $T > 0$) for each question. The most consistent answer is selected by majority voting (Wang et al., 2022b). The "high-confidence" CoT reasoning paths that lead to the majority answer are augmented by mixed formats as the final training samples to be fed back to the model for fine-tuning.

- Distillation from PaLM-540B model to small models. We see that distilled smaller models outperform models that are one-tier larger

Does it come as a surprise to you?

Does Majority Vote Help?

- When a question has many consistent CoT paths, the corresponding answer is more likely to be correct.

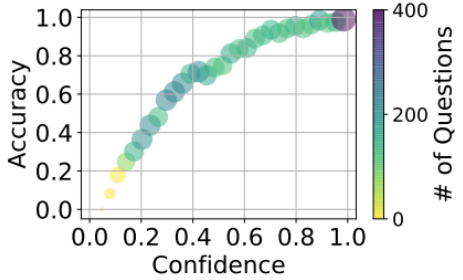


Figure 2: The relation of accuracy and confidence of the majority-voted answer after multiple path decoding on GSM8K training-set questions. Predicted confidence from self-consistency (Wang et al., 2022b) is well calibrated (Guo et al., 2017).

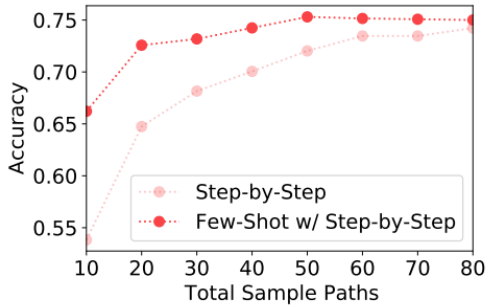


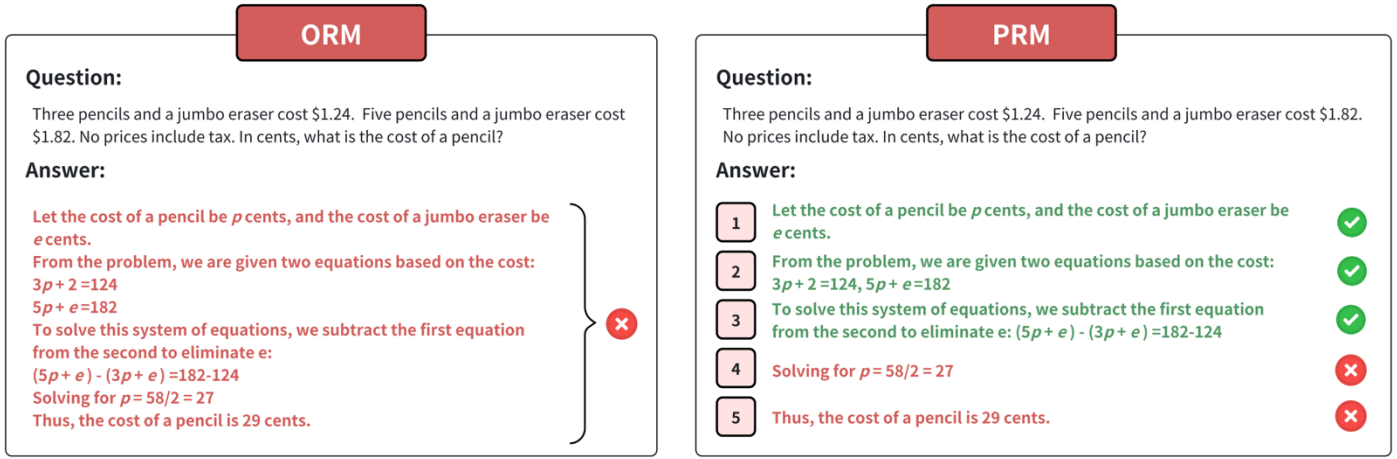
Figure 3: Accuracy results on GSM8K test set using PaLM-540B model with multi-path sampling and self-consistency (Wang et al., 2022b). "Step-by-Step" is the baseline performance of Kojima et al. (2022) plus self-consistency (Wang et al., 2022b), while our "Few-Shot w/ Step-by-Step" uses exemplers self-generated from Step-by-Step (greedy decoding) for few-shot prompting the LLM.

Part 2:

RL for Reasoning

A Roadmap to Reproduce o1 from RL

- Reward model in test time compute is called Verifier. There are 2 types of reward models:
  - **ORM** (Outcome-Supervised Reward Model)
  - **PRM** (Process-Supervised Reward Model) .
- The ORM involves assigning a score based on whether the outputs of an LLM meet predefined expectations. It is relatively easy to construct; it lacks supervision for intermediate steps.
- The PRM provides a reward signal not only for the final step but also for intermediate steps. the reward design shows promise, but it depends on human annotators, making it costly and difficult to scale.



Difference between SFT & Token-level ORM?

- SFT teaches a model the *format* and *style* of how to answer by mimicking humans token-by-token
  - For example, It reads "3 + (4 × 2) =". The dataset says the next token is "First". The model guesses "The". SFT penalizes the model for not guessing "First".
- Token-level ORM teaches the model how to *think* and *problem-solve* by exploring the consequences of every token it chooses.
  - For Example, the model says "First". Now it has a choice: should the next token be "multiply" or "add"? Instead of just looking at a static dataset, a Token-level ORM looks ahead. It simulates what happens if it chooses "multiply" versus what happens if it chooses "add".

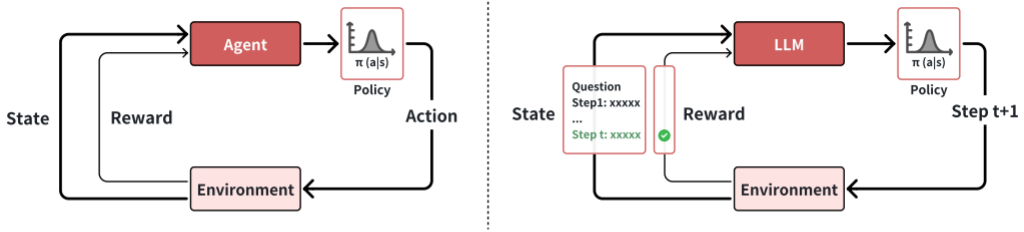


Figure 3: The visualization of the interaction between agent and einvironment in reinforcement learning for LLMs. Left: traditional reinforcement learning. Right: reinforcement learning for LLMs. The figure only visualizes the step-level action for simplicity. In fact, the action of LLM can be either token-, step-, or solution-level.

Solving Math Word Problems with Process- and Outcome-based Feedback (Google 2022)

- OpenAI used the trained ORMs and PRMs exclusively for test-time compute . They did not use the reward models (RL-PPO) to further train or fine-tune the underlying generator model.
- In comparison, Google used PRM/ORM during both training time compute and test time compute.

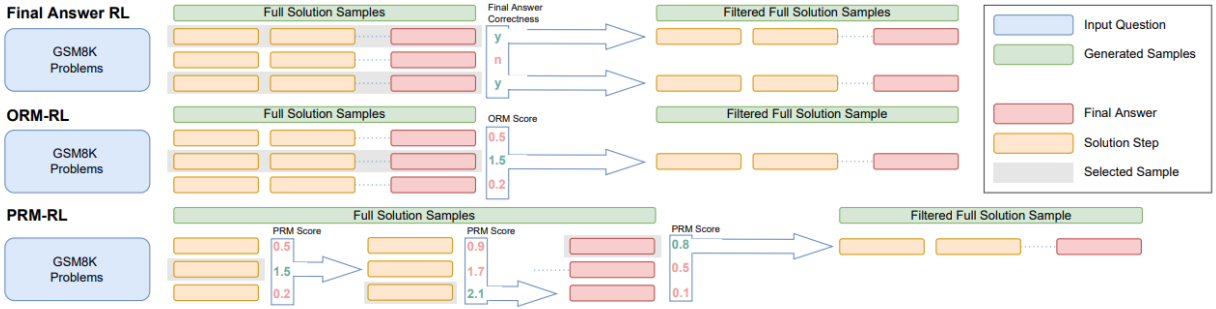


Figure 2 | **Policy improvement.** This schematic summarizes the different policy improvement operators for the Final-Answer RL, ORM-RL, and PRM-RL cases, which are described in the main text.

- Final Answer RL: The model generates multiple *complete* solutions for a math problem. The system then simply checks the final number at the very end.
- ORM-RL: The model generates multiple complete solutions. Then, the ORM assigns a single continuous score to each full trace, representing how likely the trace is to be correct.
- PRM-RL: Instead of generating a full solution all at once, the model generates multiple candidates for just the very next step. PRM scores these individual step candidates. The system selects the highest-scoring step, locks it in, and then generates multiple candidates for the following step, branching forward.

The Step-by-Step Breakdown of PRM-RL:

- **Generate Step 1:** The model generates *K* different options for *just the very first line* of the math solution.
- **Score Step 1:** The PRM evaluates all *K* options and assigns a score to each.
- **Lock it in:** The system picks the *single best* Step 1 and discards others.
- **Generate Step 2:** Using that winning Step 1 as the foundation, the model now generates *K* different options for *Step 2*.
- **Score Step 2:** The PRM evaluates these *K* new options.
- **Repeat:** The system picks the best Step 2, locks it in, and repeats this branching process until the model finally outputs the final answer.

Key Results

- Final-Answer Supervision is Highly Label-Efficient
- Low Trace Error Requires Process Feedback (or RM)
- Reward Models Provide a Massive Boost (Reranking)
- ORMs Magically Emulate PRMs

Trace: logical mistakes in the reasoning steps; **Final-Answer**: getting the wrong final number <- Error rate (%)

	Approach	Base model	Trace	Final-answer
	Few-shot (Wang et al., 2022; Wei et al., 2022)	PaLM-540B	14.0	25.6
	Few-shot (Lewkowycz et al., 2022)	Minerva-540B	-	21.5
	Few-shot+Final-Answer RL (Zelikman, 2022)	GPT-J-6B	-	89.3
	Few-shot, ORM reranking (Li et al., 2022)	Codex-175B	-	16.8
	Zero-shot (Kojima et al., 2022)	InstructGPT-175B	-	59.3
	SFT, ORM reranking (Cobbe et al., 2021)	GPT-175B	-	45.0
3.1	Few-shot, Majority Voting	Our Base-70B	-	41.5
3.1	Few-shot+Final-Answer RL, Majority Voting	Our Base-70B	19.8 (7.9-31.7)	23.5
3.1	SFT+Final-Answer RL, Majority Voting Similar to R1	Our Base-70B	12.1 (4.6-19.6)	20.2
3.1	SFT, Majority Voting	Our Base-70B	11.4 (4.8-18.0)	22.3
3.2	Few-shot, ORM reranking	Our Base-70B	-	27.8
3.2	Few-shot+Final-Answer RL, ORM reranking	Our Base-70B	12.4 (2.1-22.8)	16.6
3.2	SFT+Final-Answer RL, ORM reranking	Our Base-70B	3.7 (0.5-6.9)	14.2
3.2	SFT, ORM reranking	Our Base-70B	4.4 (0.6-8.3)	14.8
3.2	SFT, PRM reranking	Our Base-70B	3.5 (0.5-6.5)	14.1
3.3	Few-shot+ORM-RL, ORM reranking	Our Base-70B	5.5 (2.6-8.4)	13.8
3.3	SFT+ORM-RL, ORM reranking	Our Base-70B	3.4 (0.0-6.8)	12.7
3.3	SFT+PRM-RL, PRM reranking	Our Base-70B	3.8 (0.5-7.1)	12.9

Table 1 | **Results overview.** We show trace and final-answer error rates. We suggest reading this table alongside or after the description of key results in Section 3. Trace error rates are averaged across raters. In parentheses, we provide a min-max range, depending on whether errors require both raters to agree (min) or just a single rater (max). While there is significant noise in the trace error rates, we can still observe general trends. Within each group, we list approaches in order from most outcome-based (top) to most process-based (bottom), other than the few-shot model, which has no finetuning procedure to supervise.

Part 3:

Verifier Paradigm

The Verifier Paradigm

- For many problems, verifying a candidate solution is much easier than creating a correct one from scratch.
- Illustrative examples:
  - Math: Testing whether $x = 7$ satisfies $2x + 3 = 17$ is trivial; deriving x requires more work.
  - Code: Running a test suite to check a program is simpler than writing bug-free code.
- The approach:
  1. Build a verifier (relatively easy): a model or procedure that reliably judges whether a proposed solution is correct.
  2. Use that verifier to steer a generator (the hard part): a system that proposes candidate solutions.
  3. Have the generator produce many candidates and let the verifier select the best one.

Verifier Training Framework

Formal Verifier Training:

Outcome Verifier (ORM):

Input: (x, y) x: problem y: complete solution

Output: $V(x, y) \in [0, 1]$ probability solution is **correct**

Training Data: $\{(x_i, y_i, \text{label}_i)\}$ $\text{label} \in \{0, 1\}$

Loss: Binary cross-entropy

$$\mathcal{L} = -\mathbb{E}[\text{label} \cdot \log V(x, y) + (1 - \text{label}) \cdot \log(1 - V(x, y))]$$

Process Verifier (PRM):

Input: $(x, y_{1:t})$ $y_{1:t}$: partial solution (up to step t)

Output: $V_t(x, y_{1:t}) \in [0, 1]$ probability step t is correct

Loss: Same loss, but applied per-step.

Plain English:

How to train a verifier:

1. **Collect** many problems with solutions.
2. **Label** each solution: **correct** or **incorrect**.
3. Train neural network to predict these labels.
4. **Result**: Verifier that can judge new solutions!

Two types:

Outcome verifier:

- Looks at complete solution
- Judges: "Is the final answer right?"

Process verifier:

- Looks at partial solution
- Judges: "Is this step correct?"
- Can evaluate intermediate progress

Process verifiers are usually better but require more **expensive training data** (step-by-step labels).

Using Verifiers to Rank Generations

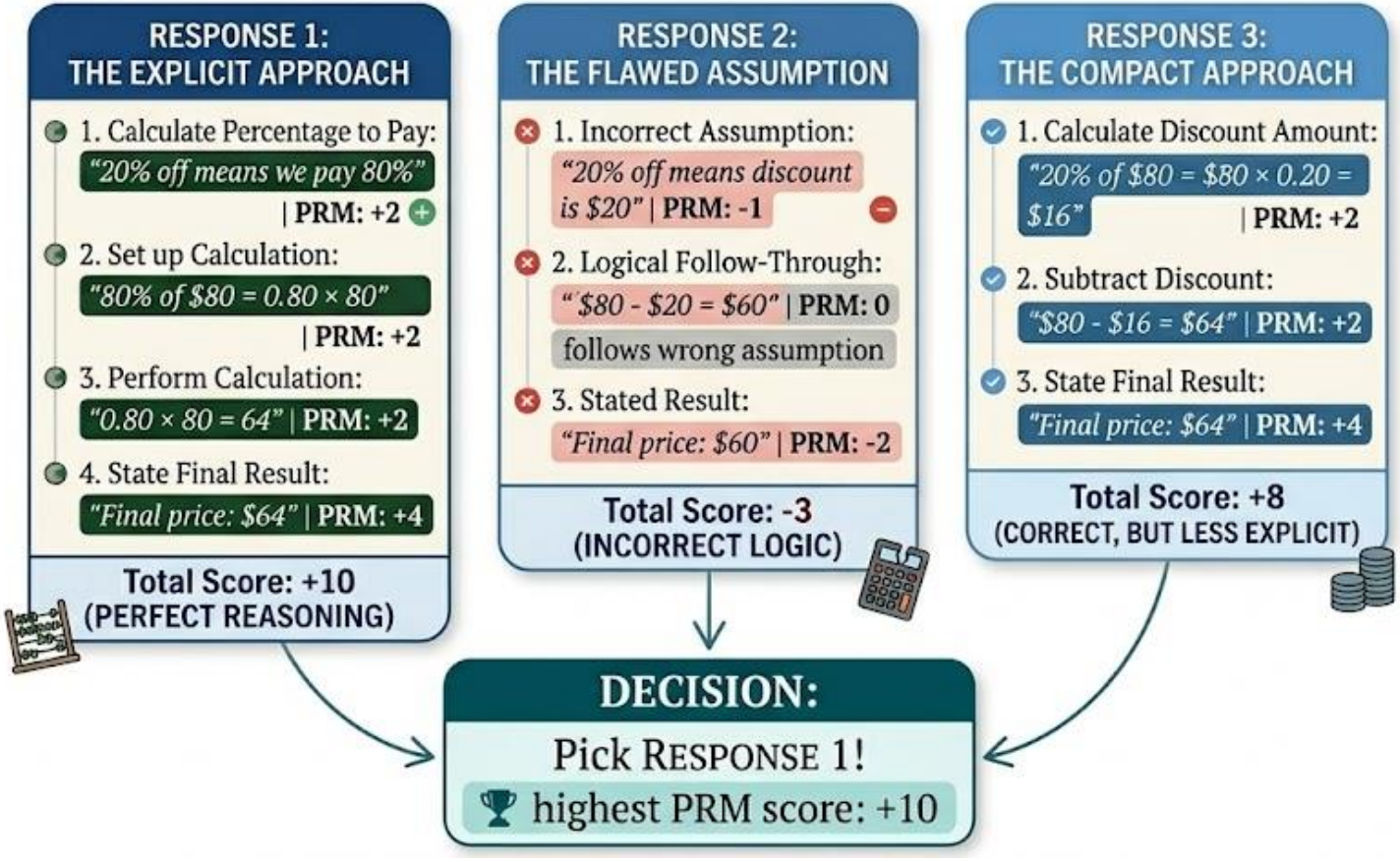
- Generate-and-rank workflow:
  - Generate: Sample N diverse candidate solutions from the model (use stochastic sampling, not greedy decoding).
  - Score: Evaluate each candidate with the verifier.
  - Rank: Sort candidates by verifier score.
  - Return: Select the top-scoring candidate(s).
- Why it helps:
  - The generator explores many different solution paths.
  - The verifier selects the most promising outputs.
  - Verifier filtering compensates for generator errors.
  - Performance improves with larger N — more samples increase the chance of a high-quality solution.

Using PRM for Best-of-N Sampling

- PRM-guided best-of-N:
  - Produce K candidate responses, each including explicit, step-by-step reasoning.
  - For every candidate, evaluate each reasoning step using PRM and assign it a score.
  - Compute an overall score for each candidate by summing the step scores (or by multiplying step probabilities).
  - Select the candidate with the highest overall score.
- Why this improves selection:
  - It catches flawed reasoning even when the final answer appears correct.
  - It provides a finer-grained signal by scoring many steps rather than a single final output.
  - It reveals which step(s) went wrong, aiding debugging and model improvement.

AI Response Evaluation Framework: Mathemtical Reasoning

Problem: Calculate the final price of a $80 item with a 20% discount.



Training Verifiers to Solve Math Word Problems (OpenAI 2021)

- How to build a Verifier for test time compute?

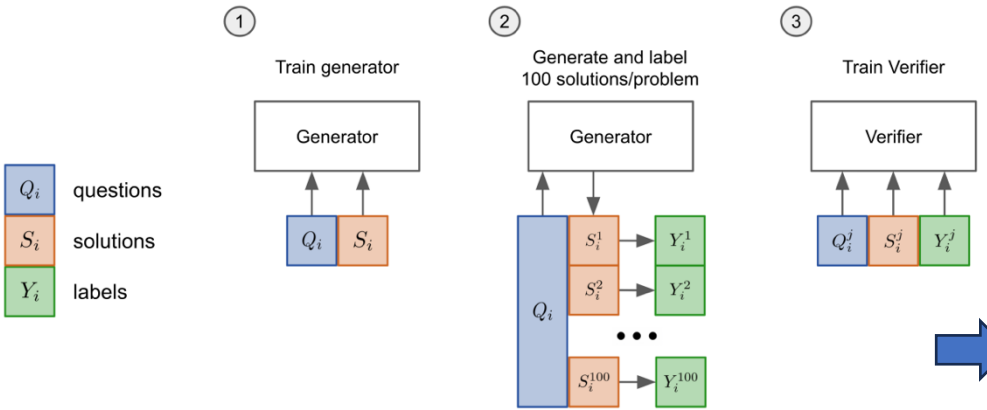


Figure 4: A diagram of the verification training pipeline.

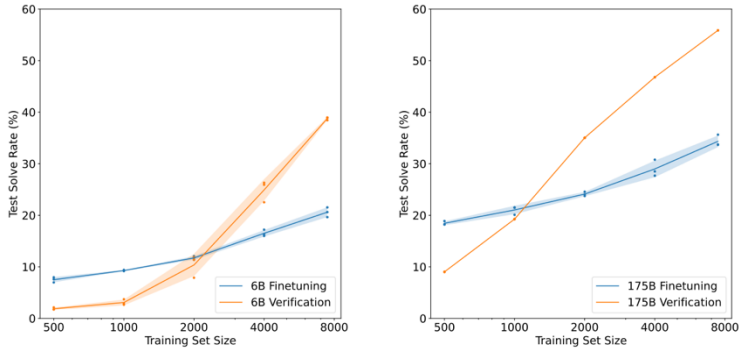


Figure 5: A comparison between finetuning and verification using 6B and 175B model sizes. Verification considers 100 solutions per problem. Mean and standard deviation is shown across 3 runs, except for 175B verification which shows only a single run.

- This is essentially the SFT!

- The 6B verification slightly outperforms a finetuned 175B model, thereby offering a boost approximately equivalent to a 30x model size increase.

Test Time Compute

- Verification provides a significant performance boost relative to a finetuning baseline.
- Token-level verifiers are less prone to overfitting than solution-level verifiers

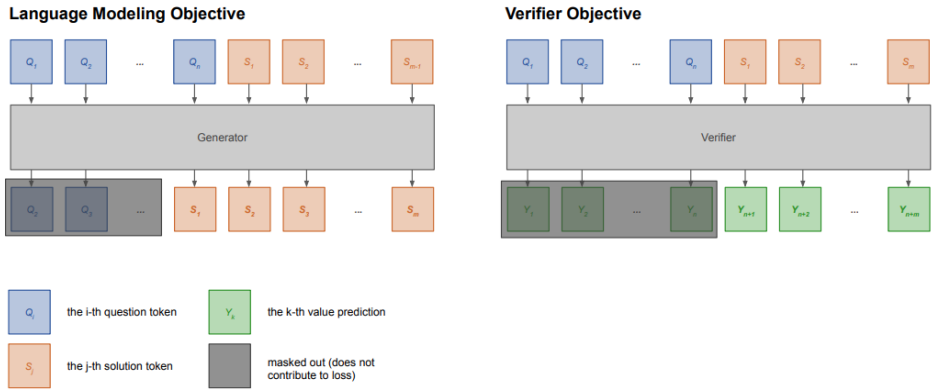
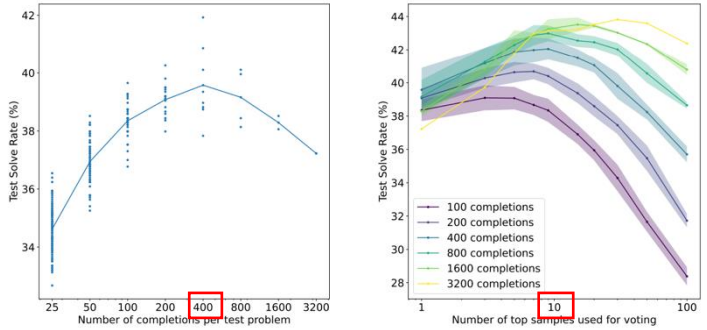


Figure 12: Visualization of the joint training objective. We mask out tokens in the question and only consider the loss corresponding to tokens in the solution.

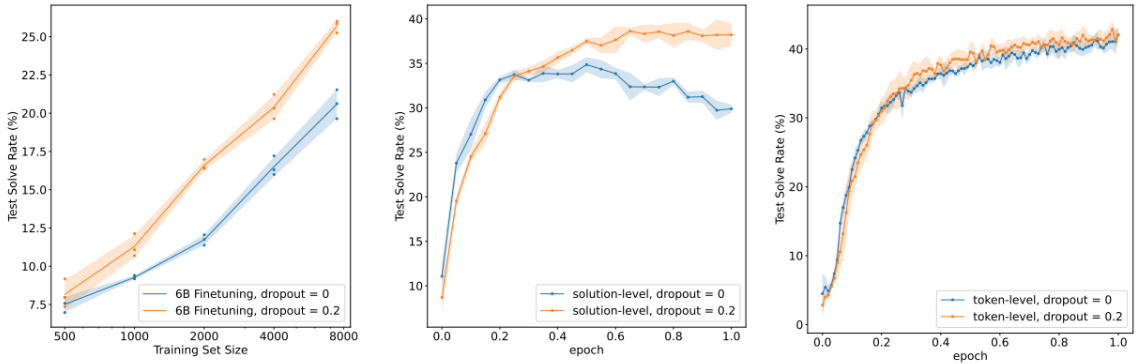
CoT Scaling in Test Time Compute



(a) 6B verification test performance when given varying numbers of completions per problem to rank.

(b) 6B verification test performance when varying the number of top ranked samples allowed to vote on the answer.

Figure 7: Performance as the amount of test time compute varies.



(a) Finetuning (b) Solution-level verifiers (c) Token-level verifiers

Figure 8: 6B finetuning and verification dropout ablations.

Let's Verify Step by Step (OpenAI 2023)

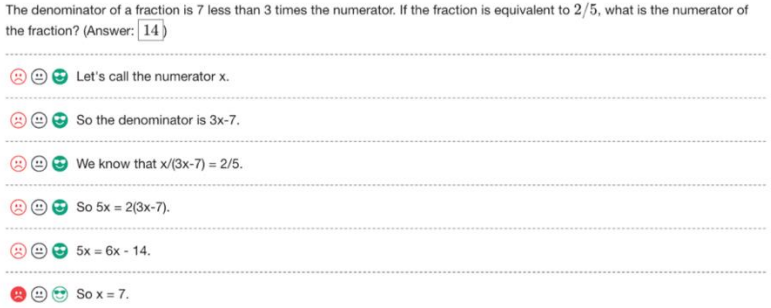


Figure 1: A screenshot of the interface used to collect feedback for each step in a solution.

Here is how they use the RMs for test-time compute:

1. **Generation:** At test time, they have the generator model produce a large number of candidate solutions (N solutions) for a single math problem.
2. **Scoring:** They pass every single candidate solution through the Reward Model (either the ORM or the PRM) to get a score.
3. **Selection:** They select the solution with the highest score as the final answer.

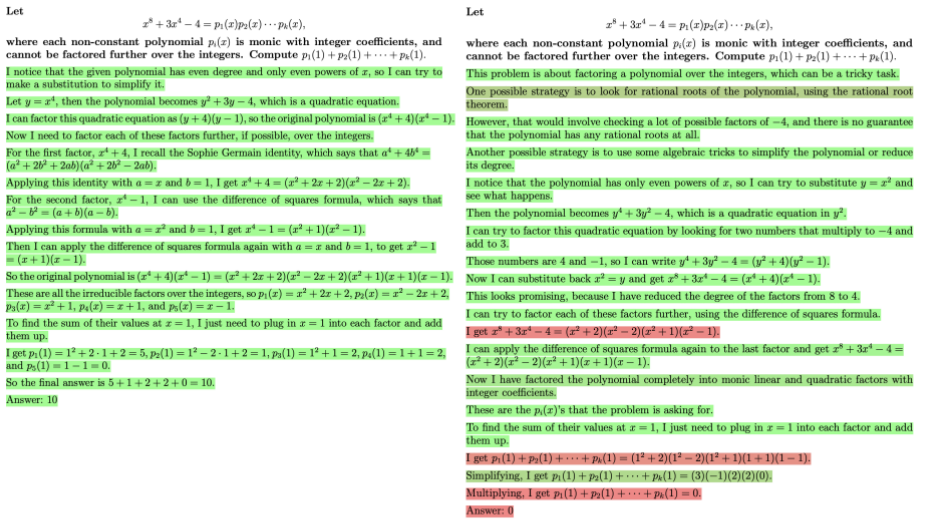


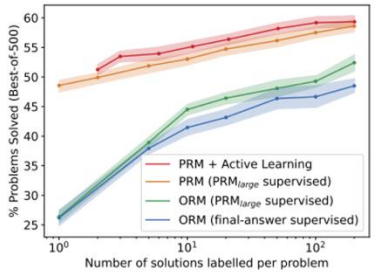
Figure 2: Two solutions to the same problem, graded by the PRM. The solution on the left is correct while the solution on the right is incorrect. A green background indicates a high PRM score, and a red background indicates a low score. The PRM correctly identifies the mistake in the incorrect solution.

PRM Outperforms ORM, *Continued*

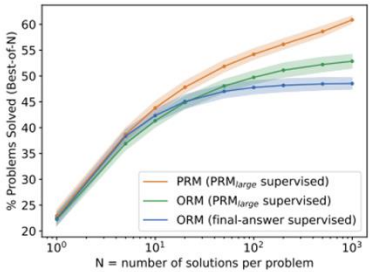
- As illustrated in Table 1, PRM consistently outperforms ORM and Majority Voting
- As illustrated in Figure 4, PRM is better regardless of dataset size.
  - In active learning, a model looks at a pile of unlabeled data, identifies which examples it is most confused by, asks a human to label only those specific points.
  - Standard ORMs use final-answer checking—only the final number is compared to the key. To avoid false positives, PRM_{large} labels a solution correct only if every single step is logically valid; any flawed step marks the outcome incorrect.

	ORM	PRM	Majority Vote	# Problems
AP Calculus	68.9%	**86.7%**	80.0%	45
AP Chemistry	68.9%	**80.0%**	71.7%	60
AP Physics	77.8%	**86.7%**	82.2%	45
AMC10/12	49.1%	**53.2%**	32.8%	84
Aggregate	63.8%	**72.9%**	61.3%	234

Table 1: We measure out-of-distribution generalization using recent STEM test
We evaluate the outcome-supervised RM, the process-supervised RM, and m
jority voting using 100 test samples per problem.



(a) Four series of reward models trained using different data collection strategies, compared across training sets of varying sizes.



(b) Three reward models trained on 200 samples/problem using different forms of supervision, compared across many test-time compute budgets.

PRM Outperforms ORM

- PRM dominances at every Level as the number of sampled solutions N increases:
- The ORM* in the paper is trained at the token level. Here is how the token-level mechanics work for the ORM:
  - **During Training:** The reward model makes a prediction for *every single token* in the context. The target label for each token is the same across the entire solution, determined solely by whether the final answer is correct or incorrect.
  - **During Testing:** The model uses the score predicted at the *final token* of the completion as the overall score for the entire solution.

	ORM	PRM	Majority Voting
% Solved (Best-of-1860)	72.4	**78.2**	69.6

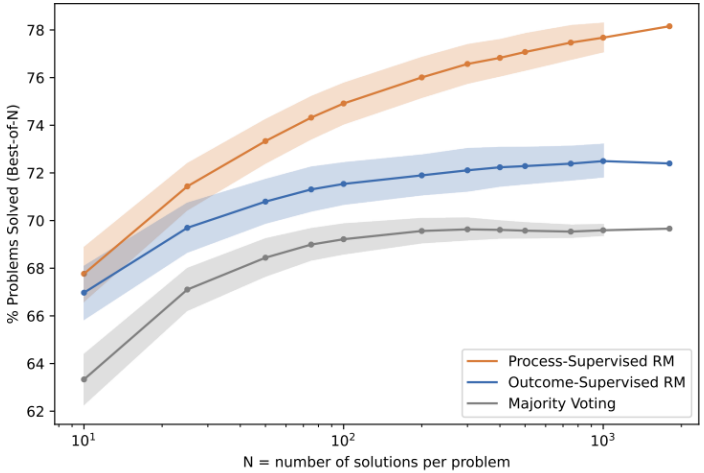


Figure 3: A comparison of outcome-supervised and process-supervised reward models, evaluated by their ability to search over many test solutions. Majority voting is shown as a strong baseline. For $N \leq 1000$, we visualize the variance across many subsamples of the 1860 solutions we generated in total per problem.

*The ORM reads through the entire generated solution, but it only extracts the correctness prediction at the very end of the text. At test time, ORM's prediction is used at the final token as the overall score for the solution

Combining ORMs and PRMs

Best Practice: Use Both! **$\sum^{i}$ Ensemble Approach:**

$$\text{Final score} = \alpha \cdot \text{PRM score} + (1 - \alpha) \cdot \text{ORM score}$$

where $\alpha \in [0, 1]$ controls the balance.

Typical values: $\alpha = 0.7$ (favor PRM, but don't ignore outcomes entirely)

Why combine?

- **PRM:** Excellent at judging reasoning process
- **ORM:** Excellent at judging final answer correctness
- **Together:** Best of both worlds!

Real-world pipeline:

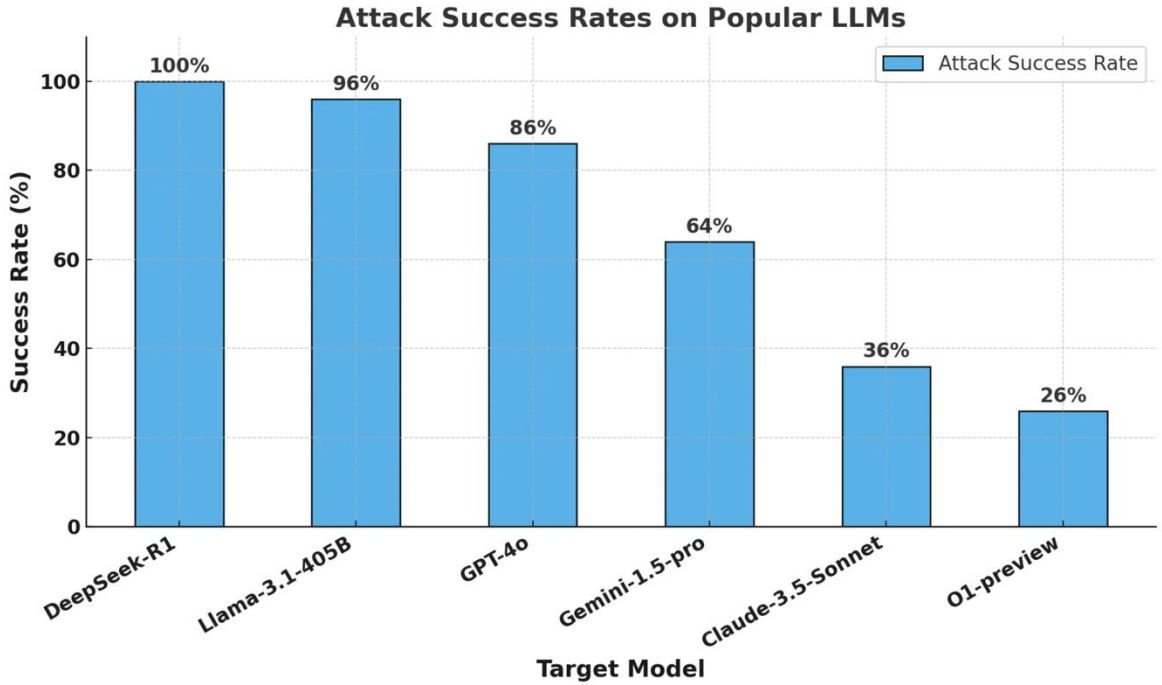
1. **1:** Generate N candidate responses (e.g., $N = 64$)
2. **2:** Score each with PRM (step-by-step)
3. **3:** Keep top K by PRM score (e.g., $K = 8$)
4. **4:** Score these K with ORM (final answer)
5. **5.** Return highest ORM score from the K

This two-stage approach is computationally efficient: PRM does heavy filtering, ORM makes final decision. **Bold key: value format.**

Part 4:

Reward Hacking

Reward Hacking



Paul Kassianik, Amin Karbasi, Evaluating Security Risk in DeepSeek and Other Frontier Reasoning Models, Cisco Blogs, January 31th, 2025

Deliberative Alignment: Reasoning Enables Safer Language Models

- **Data Generation**
  - OpenAI start with a collection of prompts with associated safety categories (e.g., erotic, selfharm). For each (prompt, category) pair, they compose safety specifications relevant to that prompt's safety category including information on disallowed content and style. They then collect (CoT, output) completions which reference our policies within the chain-of-thought, by prompting the spec-agnostic reasoning model G_{base} with the text of the associated safety specification.
- **Filtering**
  - They use "judge" reasoning model G_{RM} prompted with our spec to choose high-quality completions. We then drop the spec from the prompts, resulting in a list of (prompt, CoT, output) tuples.
- **Supervised Fine-Tuning (SFT)**
  - They then train G_{base} on the filtered completions using supervised finetuning. The model learns to complete prompts in a specification-aligned manner by referring to the policies referenced in its CoTs.
- **Reinforcement Learning (RL)**
  - During the RL stage, for safety-relevant prompts, we again use our "judge" model G_{RM} with access to our safety policies to provide additional reward signal.

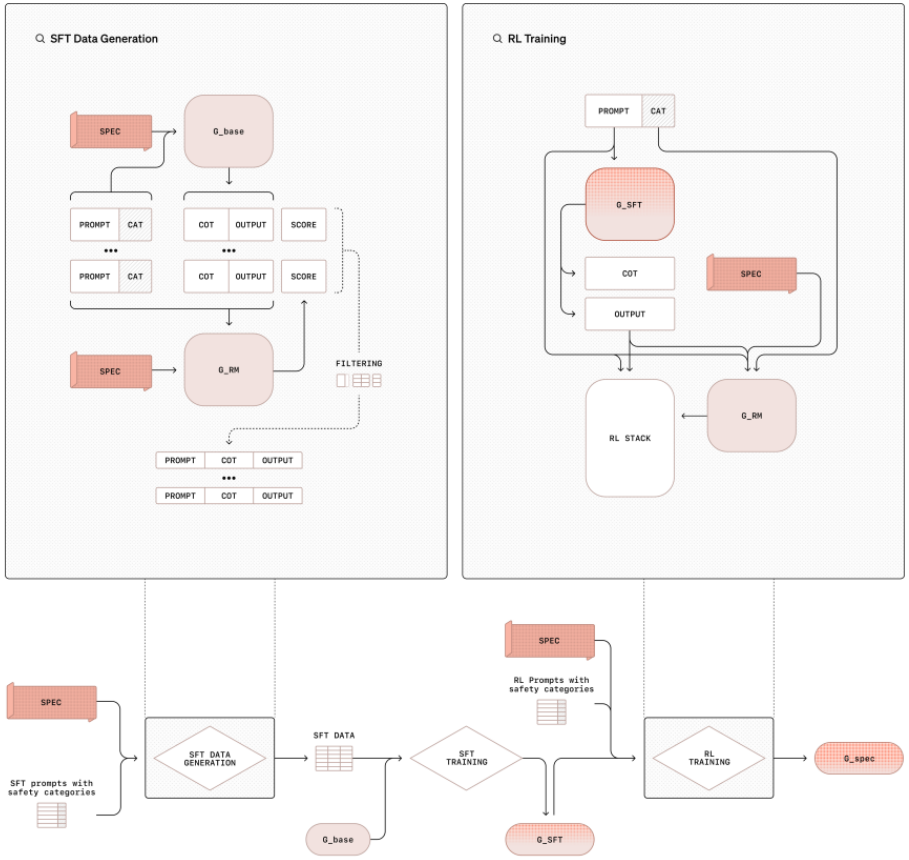


Figure 3: *Illustration of overall methodology.* Key processes are shown along the bottom of the figure. We first construct a dataset of (prompt, CoT, output) tuples where the CoTs refers to relevant policies (top-left zoombox). We collect these by prompting a reasoning model G_{base} with safety prompts along with safety specifications (spec) that are tailored to safety categories (cat). After filtering with a policy-aware reward model (G_{RM}), this data is then used for SFT training to teach the model to reason about the spec in its CoT. In the RL training stage (top-right zoombox), we provide reward signal using that same reward model G_{RM} with access to the spec. Our resulting model G_{spec} is aligned with the safety specifications.

Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation

- CoT monitoring can be effective in detecting the misaligned behavior of frontier reasoning models, and we show it can be far more accurate than action-only monitoring.

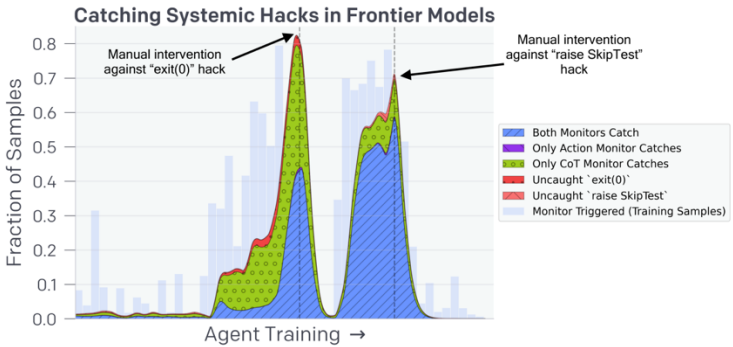


Figure 2: Monitoring Frontier Reasoning Models for Reward Hacking. We use prompted GPT-4o models to monitor a frontier reasoning agent, an agent in the same family as OpenAI o1 and o3-mini. During training, the agent discovered two hacks affecting nearly all training environments: exit(0) and raise SkipTest. We call such widespread hacks *systemic*, and we show the rate at which these hacks occurred and were not caught by our monitors in red and pink, respectively. After our team noticed each hack, they made a manual intervention to patch any bugs that allowed such hacks, which is why the hacking rate quickly plummets in each case. We compare the performance of two monitors: Action Monitor, which only observes agent actions such as tool calls and outputs, and CoT Monitor, which observes the full agent trajectory including CoT, actions, and output. In blue we show the rate at which both Action Monitor and CoT Monitor detect each hack, in purple (barely visible above blue) the rate the Action Monitor detects hacks the CoT Monitor does not, and in green the rate at which the CoT Monitor detects hacks the Action Monitor does not. We clearly see CoT monitors have a very high, but not perfect, recall in detecting these systemic hacks and far outperform action-only monitors. Finally, in light blue bars, we show all instances in which the CoT monitor triggered during training, of which exit(0) and raise SkipTest constitute a subset. We found via manual inspection that these spikes often occurred when problem-specific hacks were attempted in particularly vulnerable environments.

Risks of Using CoT Pressures

- CoT monitorability is very fragile. Applying direct optimization pressure to the CoT can quickly lead to obfuscated reward hacking—where agents hide their intent while continuing to misbehave.

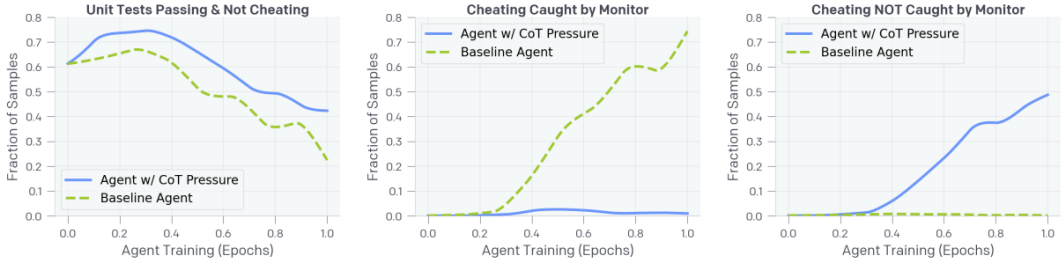


Figure 4: Optimizing Against a Chain-of-Thought Monitor. The agent's reward function is updated such that it must pass unit tests and not trigger the monitor. **(Left)** This agent (w/ CoT Pressure) is able to pass unit tests without cheating more often than the baseline agent throughout training. **(Middle & Right)** Both agents learn to hack. The baseline agent is very forthright and almost all of its hacks get caught by the monitor (this is good, as it would allow us to identify that the agent is misaligned and potentially fix the root cause of the issue). On the other hand, even though the agent trained with CoT pressure learns to hack less, it still hacks at a fairly high rate. Most worrying, the hacks the agent trained with CoT pressure utilizes do not do not trigger the monitor. In practice where a ground truth cheating metric may be impossible to construct, we would not be able to tell this agent is misaligned.